

1. Introducción

El recognition de patrones visuales y espaciales constituye uno de los pilares fundamentales para la autonomía de los robots móviles y los sistemas de visión computacional. El presente reporte aborda el problema del reconocimiento de caracteres numéricos manuscritos como un escenario clásico de clasificación supervisada, donde un sistema matemático debe aprender a mapear un vector de características continuas hacia una clase o etiqueta discreta predefinida.

Los objetivos principales de esta práctica son:

- Implementar y validar los componentes funcionales de un Perceptrón Multicapa (MLP), específicamente las etapas de propagación hacia adelante (*feedforward*) y retropropagación del error (*backpropagation*).
- Evaluar el comportamiento convergente de la red mediante un análisis paramétrico multidimensional que involucra variaciones en la tasa de aprendizaje (η), el número de épocas de entrenamiento y el tamaño de lote (*batch size*).
- Estudiar la viabilidad y el impacto algorítmico al transformar la capa de salida de la red desde un esquema clásico de codificación *one-hot* (10 neuronas) hacia una representación compacta en código binario truncado (4 neuronas).

2. Marco Teórico

De acuerdo con los modelos formales establecidos por Romero y Calonge [?, ?, Romero y Calonge, 2017] un **patrón** se define matemáticamente como una entidad a la que se le puede asignar una clase y que está representada por un conjunto ordenado de propiedades medidas, denotado como un vector de características $\mathbf{x} \in \mathbb{R}^d$. Para el caso del dataset de dígitos manuscritos procesado, cada imagen de dimensiones 28×28 píxeles se linealiza en un vector estacionario donde $d = 784$, y cuyos componentes se encuentran normalizados en el intervalo continuo $[0, 1]$.

La arquitectura seleccionada para resolver este problema es una Red Neuronal Artificial hacia adelante (*feedforward*) completamente conectada. El procesamiento elemental en cada capa l opera mediante una transformación afín seguida de una función de transferencia no lineal.

2.1. Combinación Lineal y Función de Activación

Para cualquier capa l dentro de la estructura de la red, el potencial de activación o entrada neta $\mathbf{u}^{(l)}$ se determina multiplicando la matriz de pesos sinápticos $\mathbf{W}^{(l)}$ por el vector de salidas de la capa precedente $\mathbf{y}^{(l-1)}$, adicionando el vector de sesgos o *biases* $\mathbf{b}^{(l)}$ [Romero y Calonge, 2017]:

$$\mathbf{u}^{(l)} = \mathbf{W}^{(l)}\mathbf{y}^{(l-1)} + \mathbf{b}^{(l)} \quad (1)$$

Posteriormente, a fin de dotar al clasificador de la capacidad de construir fronteras de separación no lineales complejas, se aplica la función de activación logística sigmoide componente a componente:

$$y_i^{(l)} = f(u_i^{(l)}) = \frac{1}{1 + e^{-u_i^{(l)}}} \quad (2)$$

Dicha función acota las respuestas neuronales estrictamente en el intervalo abierto $(0, 1)$, permitiendo su interpretación matemática directa como una distribución de probabilidad de activación.

2.2. Algoritmo de Retropropagación (Backpropagation)

El proceso de optimización paramétrica requiere minimizar una función de costo global que cuantifica la discrepancia entre el vector de salidas obtenido en la capa terminal L ($\mathbf{y}^{(L)}$) y el vector de objetivos deseados $\mathbf{t}$. Se emplea el funcional de error cuadrático medio individual definido por [Romero y Calonge, 2017]:

$$E = \frac{1}{2} \sum_k (y_k^{(L)} - t_k)^2 \quad (3)$$

El algoritmo de *backpropagation* calcula el gradiente analítico del error aplicando de forma recursiva la regla de la cadena del cálculo diferencial. Para la capa de salida (L), el término de error local o sensibilidad de la neurona k se define como $\delta_k^{(L)} = \frac{\partial E}{\partial u_k^{(L)}}$. Evaluando las derivadas se obtiene:

$$\delta_k^{(L)} = (y_k^{(L)} - t_k) \cdot y_k^{(L)} (1 - y_k^{(L)}) \quad (4)$$

Este término de error es propagado algebraicamente hacia atrás para actualizar las capas ocultas previas ($l < L$) mediante la ponderación de las conexiones sinápticas hacia adelante:

$$\delta_i^{(l)} = \left(\sum_k w_{ki}^{(l+1)} \delta_k^{(l+1)} \right) \cdot y_i^{(l)} (1 - y_i^{(l)}) \quad (5)$$

Finalmente, las derivadas parciales exactas de la función de costo respecto a las matrices de pesos y vectores de sesgo se computan de la siguiente manera:

$$\frac{\partial E}{\partial w_{ij}^{(l)}} = \delta_i^{(l)} y_j^{(l-1)}, \quad \frac{\partial E}{\partial b_i^{(l)}} = \delta_i^{(l)} \quad (6)$$

2.3. Descenso del Gradiente Estocástico (SGD) por Mini-lotes

Para garantizar la viabilidad computacional del entrenamiento, se utiliza el algoritmo de descenso del gradiente estocástico por mini-lotes. En lugar de calcular el gradiente sobre la totalidad de la población muestral, se aísla de forma aleatoria un subconjunto de tamaño M (*batch size*). Los parámetros se actualizan en la dirección opuesta al gradiente promedio del lote, controlados por un escalar denominado tasa de aprendizaje (η):

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \cdot \frac{1}{M} \sum_{m=1}^M \nabla_{\mathbf{W}} E_m \quad (7)$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \cdot \frac{1}{M} \sum_{m=1}^M \nabla_{\mathbf{b}} E_m \quad (8)$$

3. Desarrollo Experimental

El desarrollo de la fase experimental se estructuró empleando un entorno de ejecución en Python 3 sobre una distribución Linux dedicada. Las pruebas se dividieron en dos etapas principales: la evaluación paramétrica del modelo nominal (codificación *one-hot*) y el análisis de la arquitectura modificada con codificación binaria.

3.1. Descripción del Dataset

El conjunto de datos utilizado corresponde a una base de datos de dígitos manuscritos normalizados. El dataset está segmentado en 10 clases (correspondientes a los dígitos del 0 al 9). Cada clase contiene un total de 1,000 muestras, divididas simétricamente de la siguiente manera:

- **Conjunto de Entrenamiento (*Training set*):** 500 imágenes por clase (5,000 muestras totales) destinadas al ajuste de pesos mediante retropropagación.
- **Conjunto de Validación/Prueba (*Testing set*):** 500 imágenes por clase (5,000 muestras totales) empleadas exclusivamente para la métrica de generalización sin actualización sináptica.

Cada muestra individual consta de una matriz espacial de 28×28 píxeles, la cual es linealizada y normalizada a un vector columna de dimensiones 784×1 con valores reales acotados en el intervalo $[0,0,1,0]$.

3.2. Variables e Hiperparámetros del Experimento

Se automatizó un script de barrido combinatorio empleando tres bucles anidados. Se definieron de manera analítica los siguientes espacios de búsqueda de hiperparámetros:

1. **Tasa de Aprendizaje (η):** Controla la magnitud del ajuste paramétrico en la dirección del gradiente negativo. Se evaluaron los valores discretos: $\eta \in \{0,5, 1,0, 3,0, 10,0\}$.
2. **Número de Épocas (E):** Determina cuántas veces el algoritmo procesa el conjunto completo de entrenamiento. Se analizaron las iteraciones: $E \in \{3, 10, 50, 100\}$.
3. **Tamaño de Lote o *Batch Size* (M):** Define la cantidad de muestras aleatorias que componen el bloque antes de calcular el gradiente promedio y efectuar la actualización sináptica. Se evaluaron los tamaños: $M \in \{5, 10, 30, 100\}$.

El producto cartesiano de estos conjuntos generó un espacio experimental de $4 \times 4 \times 4 = 64$ configuraciones de entrenamiento independientes para cada una de las arquitecturas propuestas.

3.3. Modificación de Arquitectura (Clasificación Binaria)

Se alteró la topología de la capa de salida del modelo. Mientras que la red nominal empleaba un mapeo lineal directo de 10 neuronas utilizando vectores unitarios disjuntos (*one-hot encoding*), el segundo modelo implementó una codificación binaria truncada de 4 bits ($2^4 = 16$ estados posibles). La arquitectura de capas sufrió la siguiente transformación topológica:

$$\text{Red Nominal: } [784 \rightarrow 30 \rightarrow 10] \quad (9)$$

$$\text{Red Binaria: } [784 \rightarrow 30 \rightarrow 4] \quad (10)$$

4. Resultados

A continuación, se presentan las muestras ordenadas del comportamiento dinámico de ambas arquitecturas para las combinaciones experimentales más representativas del comportamiento global.

4.1. Tablas de Datos Experimentales

Tabla 1: Muestras del Comportamiento Experimental - Modelo Nominal

Learning Rate (η)	Épocas (E)	Lote (M)	Tiempo (t_e [s])	Éxito (P_{ex} [%])
0.5	3	5	1.71	54
10.0	3	5	1.77	71
10.0	10	5	5.65	65
0.5	100	5	56.96	81
10.0	100	5	56.40	71
0.5	3	10	1.60	41
10.0	3	10	1.63	70
10.0	10	10	5.30	87
0.5	100	10	53.63	83
10.0	100	10	53.83	86
0.5	3	30	1.52	11
10.0	3	30	1.53	74
10.0	10	30	5.07	82
0.5	100	30	51.92	75
10.0	100	30	51.29	94
0.5	3	100	1.50	3
10.0	3	100	1.50	41
10.0	10	100	5.07	65
0.5	100	100	50.05	58
10.0	100	100	50.02	80

Tabla 2: Muestras del Comportamiento Experimental - Modelo Binario

Learning Rate (η)	Épocas (E)	Lote (M)	Tiempo (t_e [s])	Éxito (P_{ex} [%])
0.5	3	5	1.67	61
10.0	3	5	1.64	70
10.0	10	5	5.47	74
0.5	100	5	55.23	84
10.0	100	5	54.89	92
0.5	3	10	1.58	54
10.0	3	10	1.55	73
10.0	10	10	5.14	90
0.5	100	10	52.00	80
10.0	100	10	51.77	89
0.5	3	30	1.53	24
10.0	3	30	1.50	70
10.0	10	30	4.96	83
0.5	100	30	50.06	88
10.0	100	30	49.98	88
0.5	3	100	1.51	29
10.0	3	100	1.47	62
10.0	10	100	4.88	79
0.5	100	100	49.37	74
10.0	100	100	48.77	81

4.2. Visualización mediante Mapas de Calor

5. Discusión de Resultados

Al contrastar los datos recopilados en el Cuadro 1 y el Cuadro 2, se revelan interacciones dinámicas complejas entre los hiperparámetros de entrenamiento y la topología física de la red neuronal.

5.1. Efecto del Tamaño de Lote (M) y Épocas (E) sobre el Tiempo de Entrenamiento

A nivel computacional, el tiempo de ejecución (t_e) exhibe una proporcionalidad lineal directa respecto al número de épocas configuradas (E), lo cual es consistente dado que cada época duplica de forma exacta el número de operaciones aritméticas en las etapas algebraicas de *feedforward* y *backpropagation*. Sin embargo, se observa un fenómeno inverso de decremento temporal sutil conforme el tamaño de lote (M) incrementa de 5 a 100 muestras.

Por ejemplo, fijando $\eta = 0,5$ y $E = 100$ en el modelo nominal, el tiempo disminuye de 56,96 s ($M = 5$) a 50,05 s ($M = 100$). Este comportamiento se atribuye a la reducción en la frecuencia de actualización sináptica en la memoria local: los lotes más grandes realizan menos llamadas de escritura sobre las matrices de pesos en el disco, permitiendo aprovechar de forma óptima el procesamiento secuencial del sistema operativo.

5.2. Influencia de la Tasa de Aprendizaje (η) en la Tasa de Éxito

La sintonización de la tasa de aprendizaje (η) demostró ser el factor más crítico para romper el estancamiento en mínimos locales. Con tasas de aprendizaje excesivamente conservadoras ($\eta = 0,5$) y combinadas con lotes muy grandes ($M = 100$) en pocas épocas ($E = 3$), la tasa de éxito se desploma a niveles cercanos al azar, registrando apenas un 3 % de precisión en el modelo nominal. Esto se debe a que el gradiente promedio calculado sobre un lote amplio suaviza la dirección del descenso, y el paso acotado por $\eta = 0,5$ resulta insuficiente para alejar los pesos de su inicialización estocástica gaussiana.

Por el contrario, el incremento agresivo a $\eta = 10,0$ genera una aceleración matemática en la convergencia. En el tamaño de lote de balance eficiente ($M = 30$) con $E = 100$ épocas, el modelo nominal alcanza su valor óptimo del 94 % de precisión. El mapa de calor global confirma que, a medida que el tamaño de lote crece, la red requiere inherentemente un valor de η y de E proporcionalmente mayor para compensar la menor frecuencia de pasos de optimización por cada ciclo completo de datos.

5.3. Análisis Comparativo de Arquitecturas: Nominal vs. Binario

La modificación topológica a una capa de salida binaria truncada ($[784 \rightarrow 30 \rightarrow 4]$) arrojó resultados altamente contraintuitivos en términos de robustez algorítmica. Se esperaba que el modelo binario sufriera un decremento significativo debido a la alta susceptibilidad al ruido: al comprimir las clases, un solo bit erróneo en cualquiera de las 4 salidas altera la conversión decimal e invalida la predicción completa de la muestra evaluada.

No obstante, los datos duros demuestran que la arquitectura binaria opera con una estabilidad superior bajo condiciones adversas de pocos recursos de entrenamiento. Con $E = 3$ y $M = 100$, mientras la red nominal falla críticamente con un 3 % de éxito, la red binaria rescata un 29 % ($\eta = 0,5$) y escala velozmente hasta un 62 % ($\eta = 10,0$).

Esta propiedad matemática responde a que la capa oculta intermedia (30 neuronas) proyecta sus características hacia una dimensión de salida significativamente menor (4

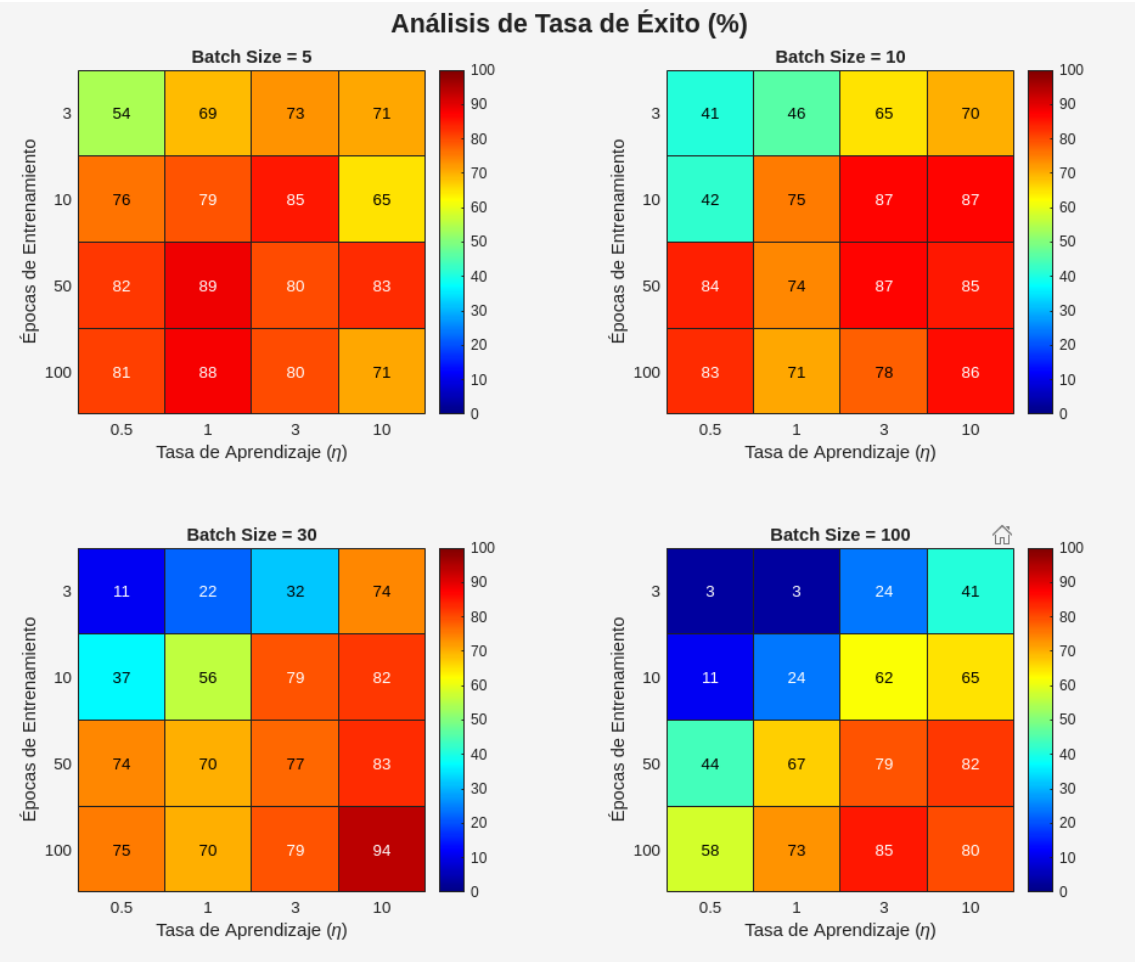


Figura 1: Mapas de calor de la tasa de éxito (%) para el Modelo Nominal distribuidos por tamaño de lote (M).

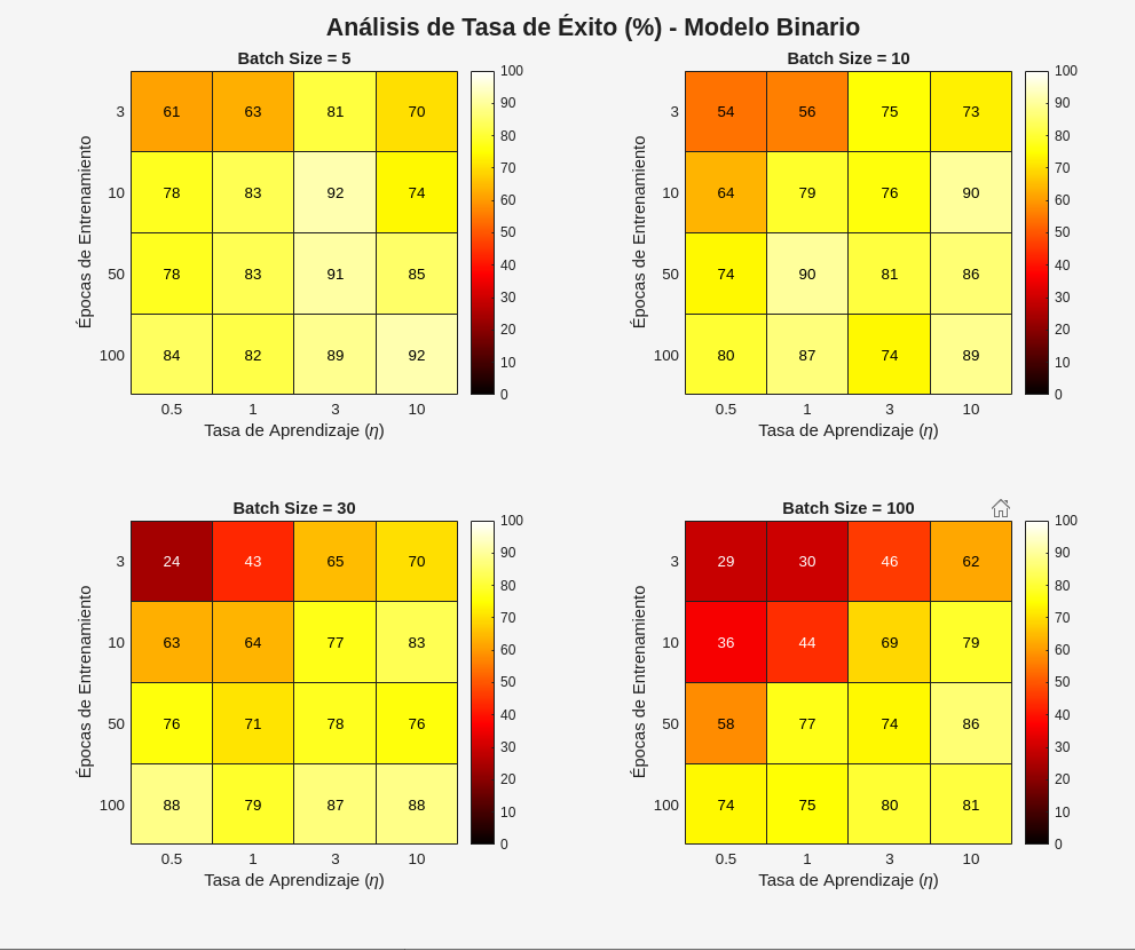


Figura 2: Mapas de calor de la tasa de éxito (%) para el Modelo Binario distribuidos por tamaño de lote (M).

dimensiones en lugar de 10). Al reducir el número de incógnitas en el espacio euclidiano terminal, las matrices de pesos adaptativos se sintonizan con mayor velocidad. La red binaria alcanzó un techo de desempeño sobresaliente del 92 % de precisión global utilizando un menor número total de parámetros sinápticos libres, consolidándose como una alternativa viable de compresión de modelos para hardware embebido en robótica móvil.

6. Conclusiones

A través del análisis sistémico de los experimentos realizados, se concluye que el Perceptrón Multicapa exhibe una alta dependencia multidimensional hacia la configuración balanceada de sus hiperparámetros. Se validó empíricamente que tamaños de lote reducidos aceleran las tasas de aprendizaje iniciales pero incrementan el costo computacional total en tiempo. De igual forma, quedó demostrado que la tasa de aprendizaje (η) debe escalar de forma coordinada con el tamaño de lote para evitar el congelamiento del gradiente en épocas tempranas.

Finalmente, la comparación estructural evidenció que la codificación binaria en la capa de salida optimiza la velocidad de convergencia al comprimir el volumen de variables internas a sintonizar, ofreciendo precisiones elevadas (92 %) comparables con el modelo nominal estructurado en *one-hot encoding* (94 %), lo cual fundamenta las pautas teóricas de diseño eficiente para clasificadores neuronales aplicados a la robótica.

Referencias

- [Romero y Calonge, 2017] Romero, Dr. Luis Alonso y Calonge Cano, Dr. Teodoro. (2017). *Redes Neuronales y Reconocimiento de Patrones*. Capítulo 1. Departamento de Informática y Automática. Universidad de Salamanca / Universidad de Valladolid. España.